Matière : **Conception et développement d'applications**
Thème : **Préparation épreuve E6 BTS SIO option SLAM**
Application 1 : **Cahier des charges techniques (trame)**

Sommaire

1. Contexte du projet....2
1.1. Présentation du projet....2
1.2. Date de rendu du projet....2
2. Besoins fonctionnels....2
3. Ressources nécessaires à la réalisation du projet....2
3.1. Ressources matérielles....2
3.2. Ressources logicielles....2
4. Gestion du projet....3
5. Conception du projet....3
5.1. Le front-end....3
5.1.1. Wireframes....3
5.1.2. Maquettes....7
5.1.3. Arborescences....9
5.2. Le back-end....10
5.2.1. Diagramme de cas d'utilisation....10
5.2.2. Diagramme d'activités....11
5.2.3. Modèle Conceptuel de Données (MCD)....11
5.2.4. Modèle Logique de Données (MLD)....12
5.2.5. Modèle Physique de Données (MPD)....12
6. Technologies utilisées....13
6.1. Langages de développement Web....13
6.2. Base de données....13
7. Sécurité....13
7.1. Login et protection des pages administrateurs....13
7.2. Hachage des mots de passe avec Bcrypt....14
7.3. Protection contre les attaques XSS (Cross-Site Scripting)....14
7.4. Protection contre les injections SQL....14

1. Contexte du projet

1.1. Présentation du projet

Votre agence web a été sélectionnée par le comité d'organisation des jeux olympiques de Los Angeles 2028 pour développer une application web permettant aux organisateurs, aux médias et aux spectateurs de consulter des informations sur les sports, les calendriers des épreuves et les résultats des JO 2028.

Votre équipe et vous-même avez pour mission de proposer une solution qui répondra à la demande du client.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard à la rentrée (Novembre).

2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

Partie publique (accessible à tous les usagers)

- consulter la **liste des sports** ;
- consulter le **calendrier des épreuves** (sport, date, heure et lieu) ;
- consulter les **résultats** des épreuves ;
- accéder à la page de **connexion administrateur**.

Partie privée (réservée aux administrateurs authentifiés)

- se **connecter** avec un login et un mot de passe ;
- gérer (ajouter, consulter, modifier, supprimer) les **administrateurs**, les **sports**, les **lieux**, le **calendrier des épreuves**, les **pays**, les **athlètes**, les **genres** et les **résultats** ;
- être protégé contre la saisie de **doublons** lors de l'ajout et de la modification des données.

Contraintes d'usage

- le site doit s'afficher correctement sur **ordinateur, tablette et smartphone** (responsive) ;
- les pages d'administration ne sont accessibles qu'après authentification.

Ces besoins sont détaillés dans le diagramme de cas d'utilisation (section 5.2.1).

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

- **Ordinateur** (PC sous Windows) doté d'au moins 8 Go de RAM, pour exécuter simultanément un serveur local, un éditeur de code et un navigateur web.
- **Connexion Internet** pour la recherche documentaire, l'accès aux environnements cloud (GitHub, Figma, InfinityFree) et le déploiement du site.

3.2. Ressources logicielles

- **Éditeur de code (IDE)** : Visual Studio Code, avec les extensions PHP, HTML, CSS et SQL.
- **Environnement serveur local** : MAMP, qui fournit les services Apache, MySQL et PHP.
- **Gestionnaire de base de données** : phpMyAdmin.
- **Outil de maquettage** : Figma ou Mockflow, pour la création des wireframes du front-end.
- **Gestion de projet** : Trello (méthode Agile Kanban).
- **Versioning et déploiement** : Git en local, GitHub pour l'hébergement du code source, InfinityFree pour la mise en ligne du site et de la base de données, et FileZilla pour le transfert des fichiers en FTP.
- **Tests** : Composer et PHPUnit, pour les tests unitaires.

Sources documentaires

- **Ressources du projet** : documents, dossier « admin-sports », trame du cahier des charges et scripts SQL fournis sur l'ENT par le professeur.
- **Back-end** : manuel officiel PHP (php.net) et documentation MySQL (dev.mysql.com).
- **Front-end** : MDN Web Docs (Mozilla) pour les standards HTML, CSS et JavaScript.
- **Sécurité** : OWASP, pour les bonnes pratiques contre les attaques XSS et les injections SQL.

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.

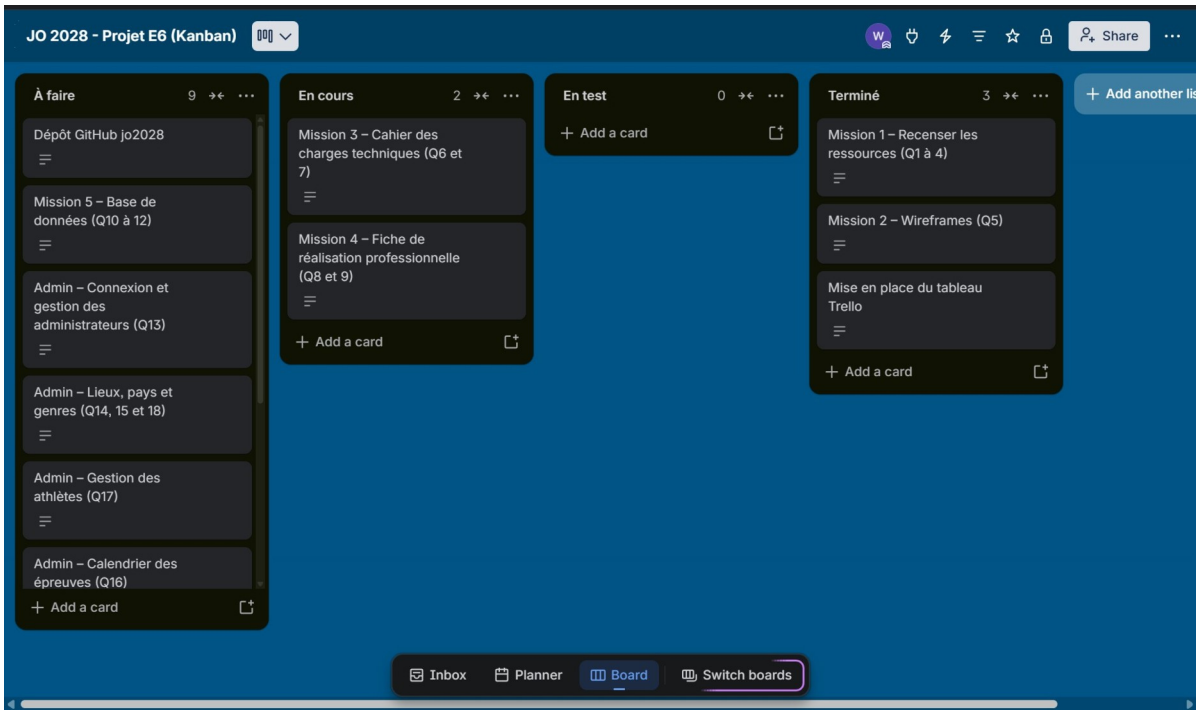


Figure 1 – Tableau Kanban du projet sur Trello

Nous travaillons également sur GitHub, plateforme de développement collaboratif.

5. Conception du projet

5.1. Le front-end

5.1.1. Wireframes

Les wireframes ci-dessous présentent la structure de chaque page, en noir et blanc, sans mise en forme graphique. Chaque page est proposée en version ordinateur (à gauche) et en version smartphone (à droite). Le site conserve une structure commune : en-tête avec menu de navigation, contenu, puis pied de page avec le logo LA28 et les anneaux olympiques.

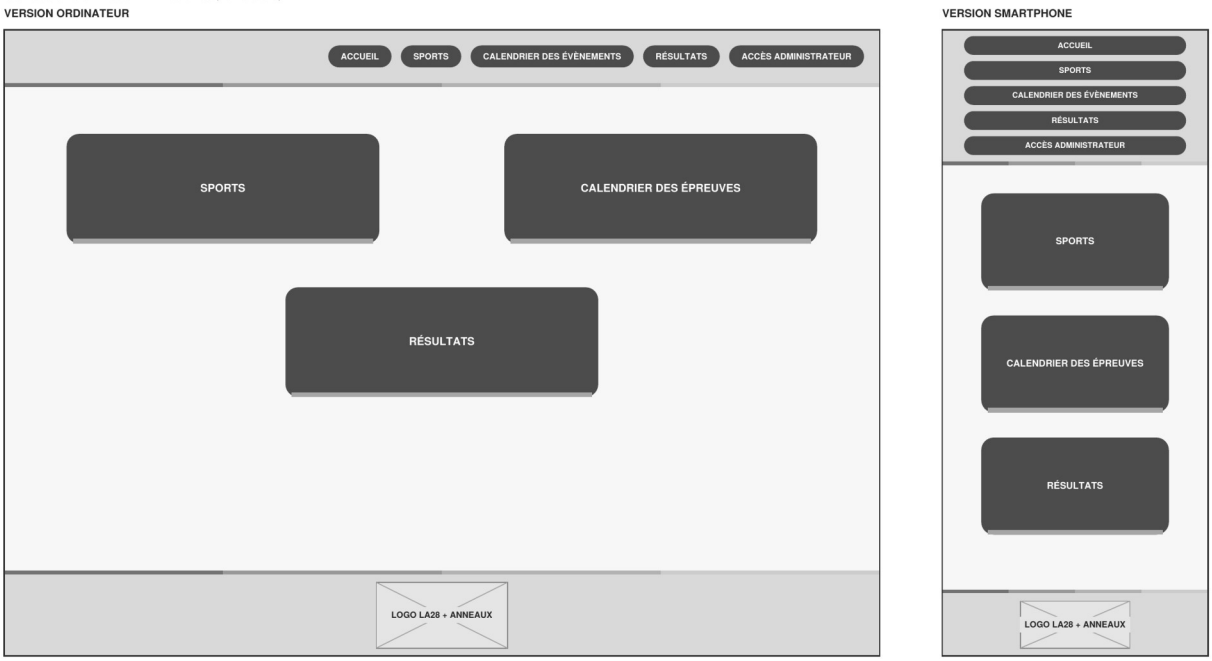
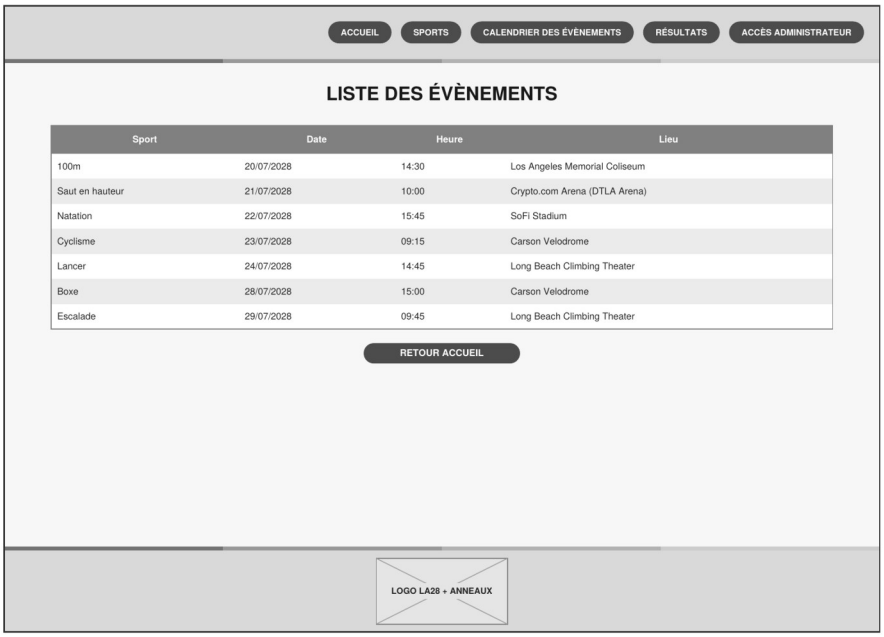


Figure 2 – Wireframe de la page d'accueil



Figure 3 – Wireframe de la liste des sports

VERSION ORDINATEUR

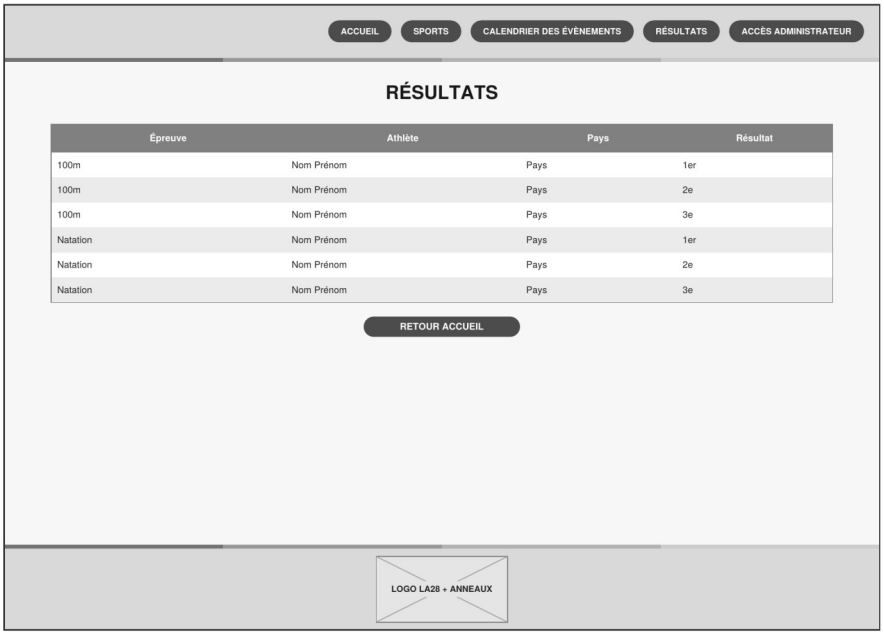


VERSION SMARTPHONE



Figure 4 – Wireframe du calendrier des évènements

VERSION ORDINATEUR



VERSION SMARTPHONE



Figure 5 – Wireframe des résultats

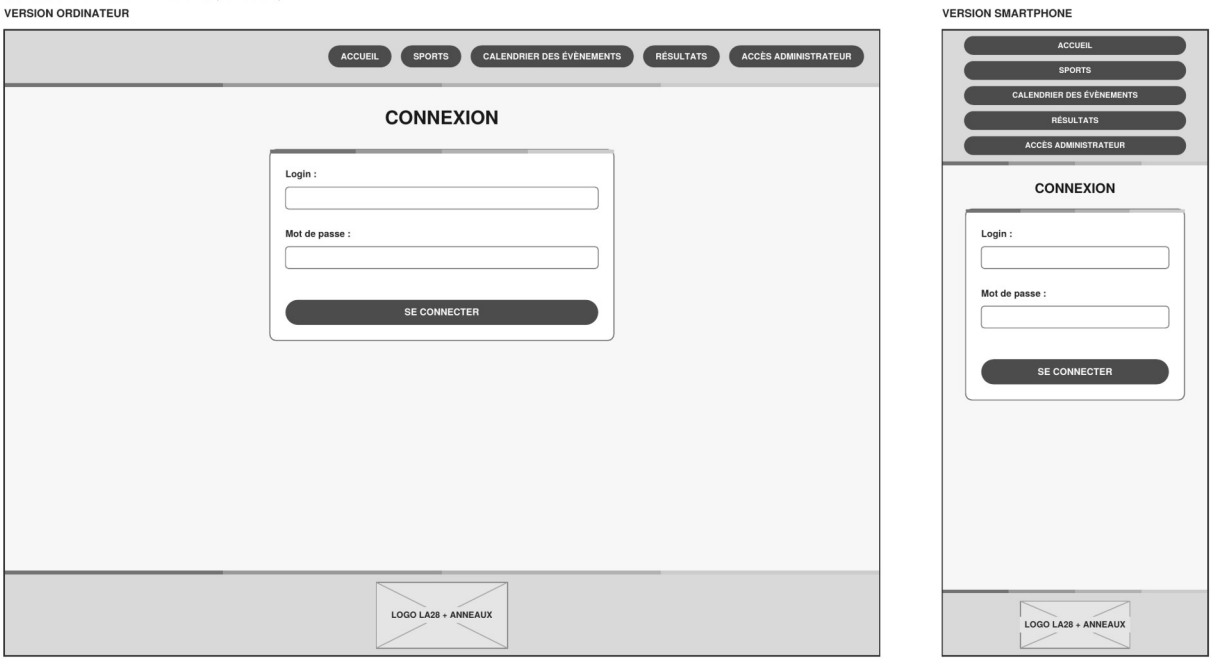


Figure 6 – Wireframe de la connexion administrateur

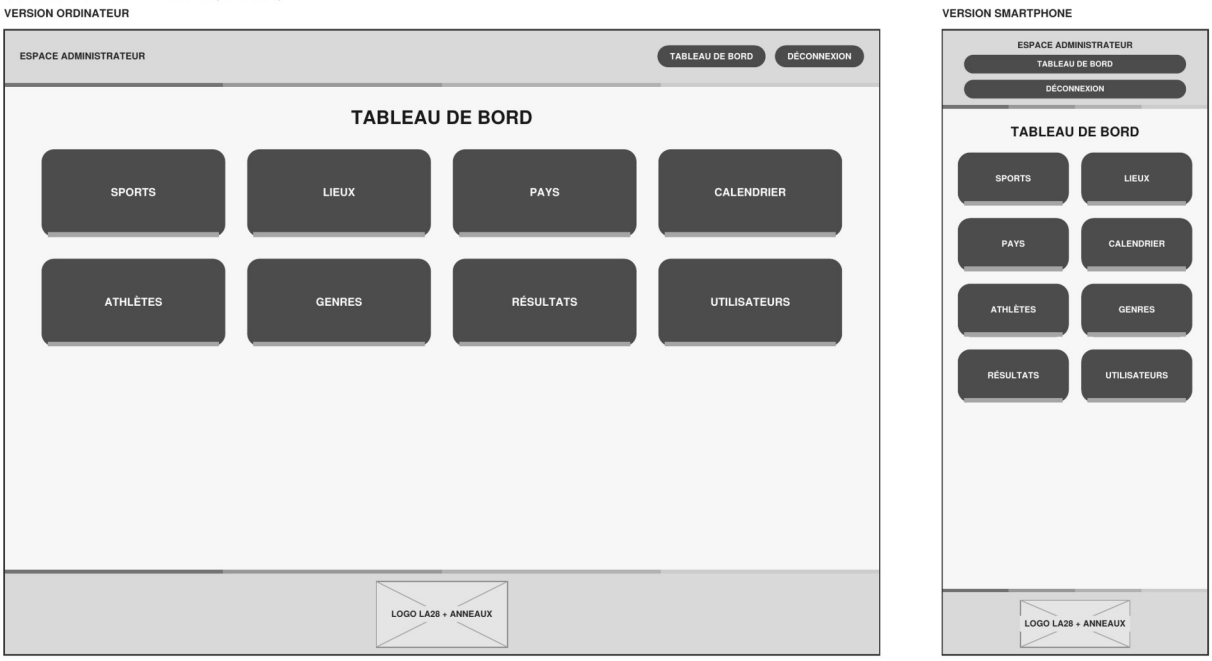


Figure 7 – Wireframe du tableau de bord administrateur



Figure 8 – Wireframe de la liste d'une entité en administration (exemple des sports)



Figure 9 – Wireframe du formulaire d'ajout et de modification (exemple des sports)

5.1.2. Maquettes

Les maquettes graphiques fournies par le client définissent la charte visuelle du site : fond sombre, boutons arrondis, bandeau aux couleurs olympiques et logo LA28 en pied de page. Elles existent en version ordinateur, tablette et smartphone.

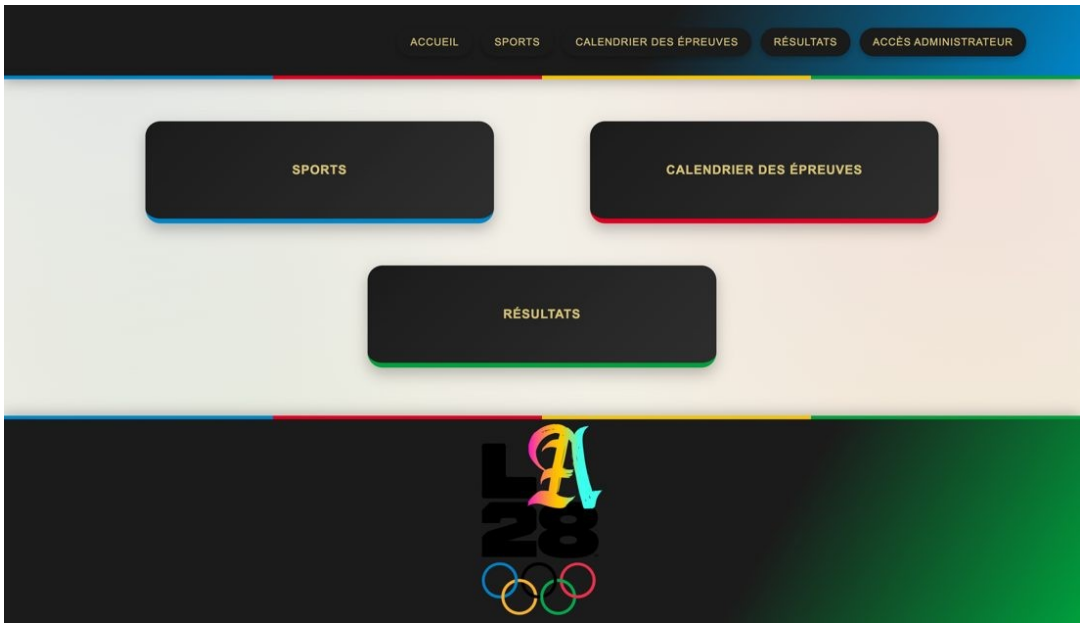


Figure 10 – Maquette de la page d'accueil (ordinateur)

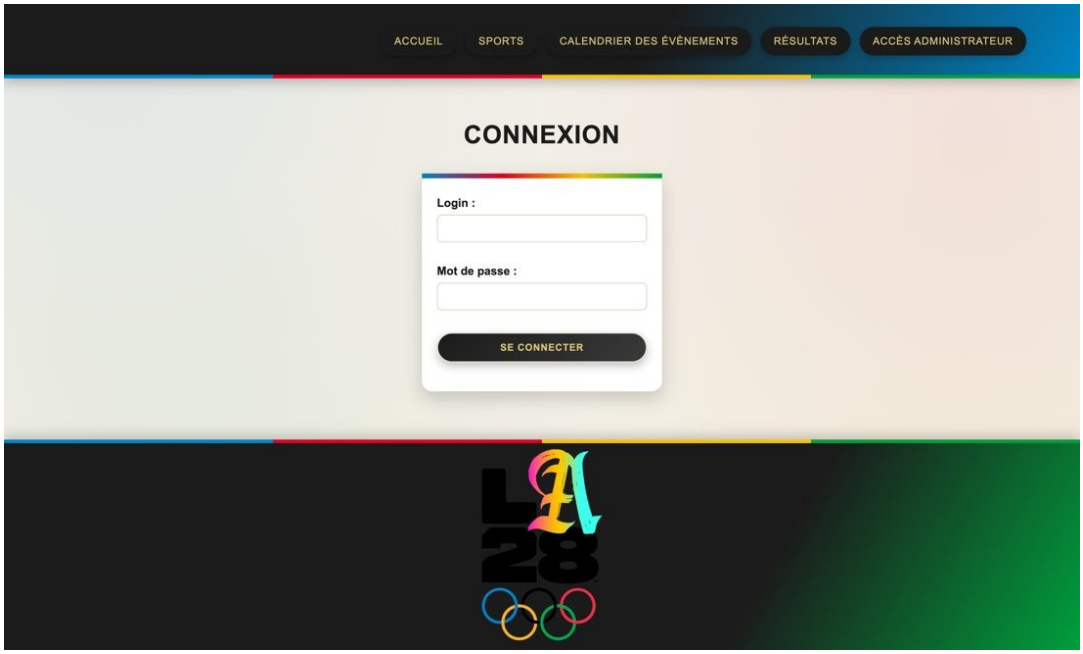


Figure 11 – Maquette de la page de connexion (ordinateur)

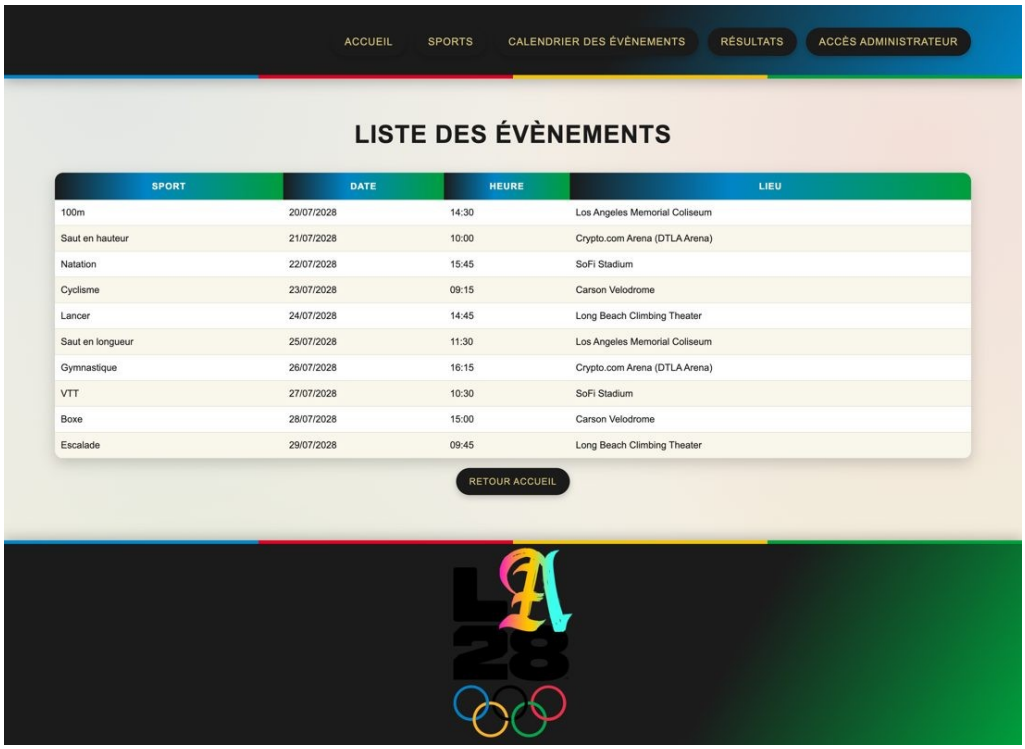
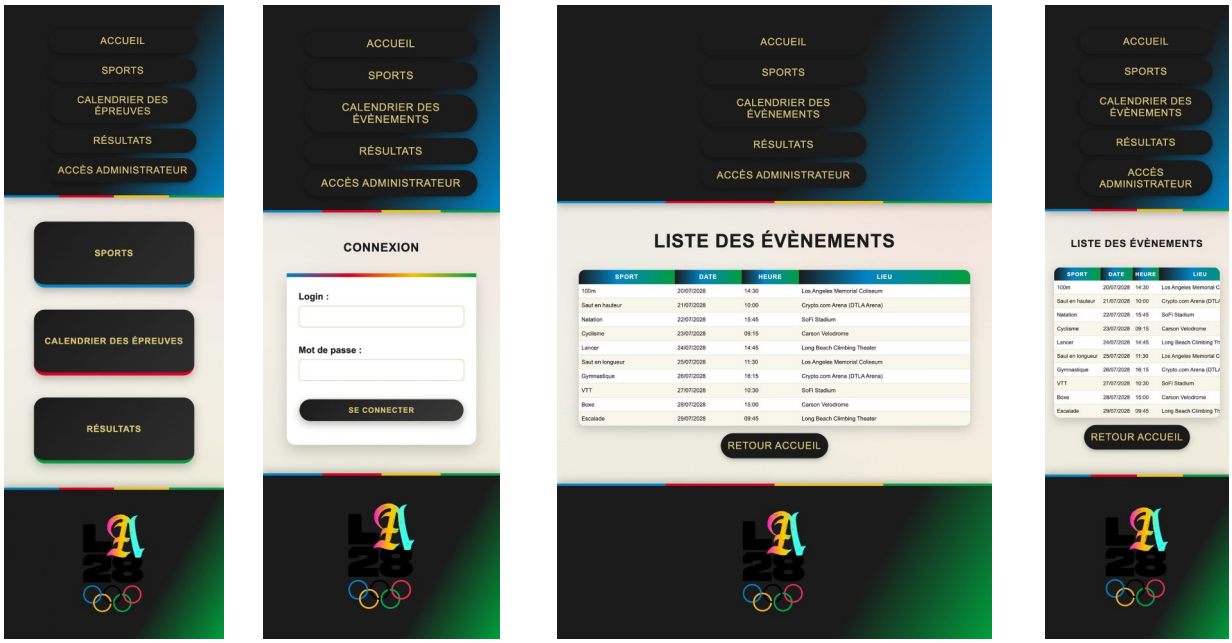


Figure 12 – Maquette du calendrier des évènements (ordinateur)



Accueil | Connexion | Calendrier (tablette) | Calendrier (smartphone)

Figure 13 – Maquettes responsive : accueil, connexion, calendrier (tablette et smartphone)

5.1.3. Arborescences

Le site est organisé en deux espaces : les pages publiques (en jaune) et les pages réservées aux administrateurs (en vert).

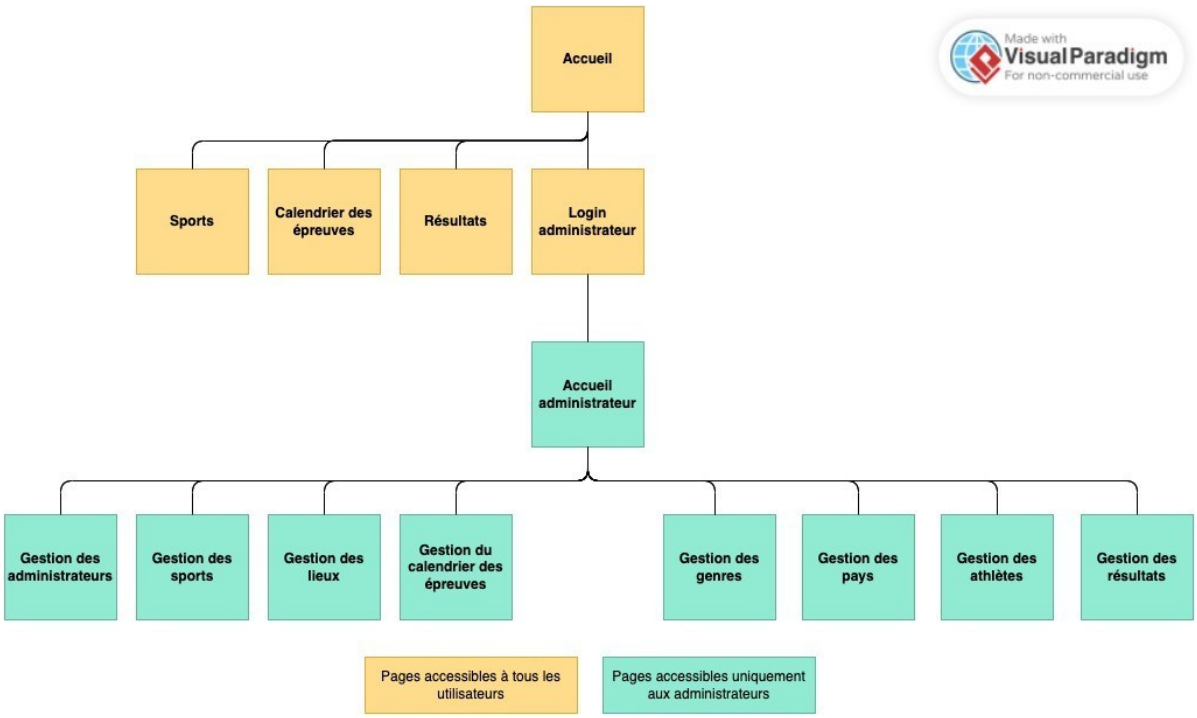


Figure 14 – Arborescence du site

- **Espace public** : accueil, sports, calendrier des épreuves, résultats et login administrateur.
- **Espace administrateur** : après authentification, l'accueil administrateur donne accès à huit pages de gestion (administrateurs, sports, lieux, calendrier des épreuves, genres, pays, athlètes et résultats).

5.2. Le back-end

5.2.1. Diagramme de cas d'utilisation

Deux acteurs interagissent avec le système : l'**usager**, qui consulte les informations, et l'**administrateur**, qui doit d'abord se connecter pour gérer les informations.

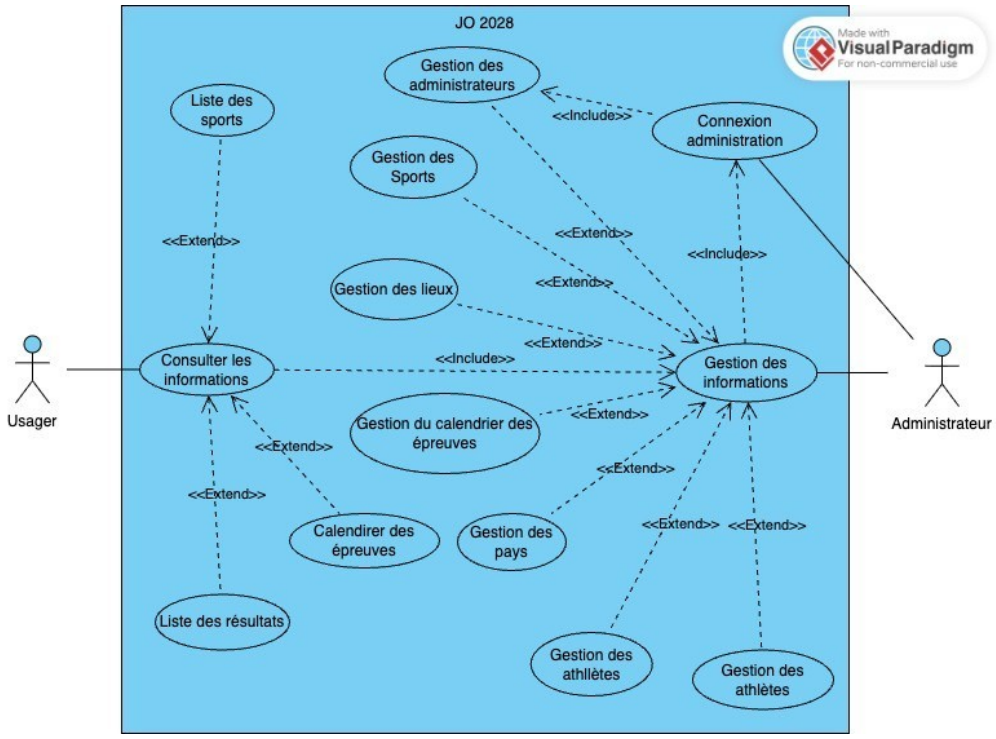


Figure 15 – Diagramme de cas d'utilisation

- **Usager** : consulte la liste des sports, le calendrier des épreuves et la liste des résultats.

- **Administrateur** : se connecte à l'espace d'administration, puis gère les administrateurs, les sports, les lieux, le calendrier des épreuves, les pays, les athlètes (et les genres, présents dans l'arborescence).

5.2.2. Diagramme d'activités

Ce diagramme décrit le scénario d'authentification d'un administrateur.

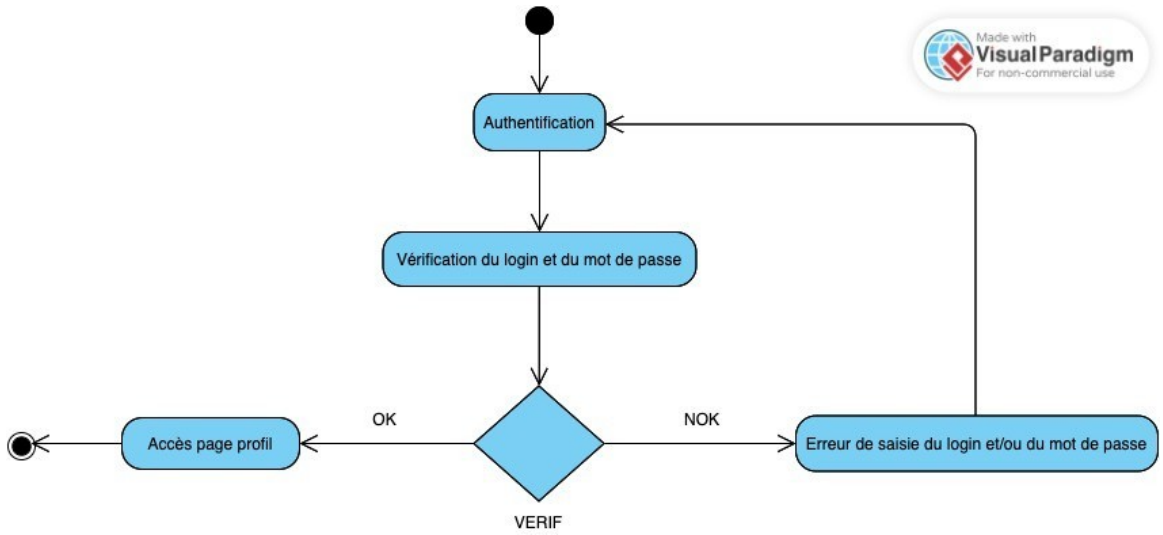


Figure 16 – Diagramme d'activités de la connexion

- L'utilisateur saisit son login et son mot de passe (authentification).
- Le système vérifie le login et le mot de passe.
- Si les identifiants sont corrects (OK), l'accès à l'espace administrateur est accordé.
- Sinon (NOK), un message d'erreur est affiché et l'utilisateur revient à l'étape d'authentification.

5.2.3. Modèle Conceptuel de Données (MCD)

Le modèle conceptuel de données comporte sept entités reliées entre elles, plus l'entité ADMINISTRATEUR, indépendante.

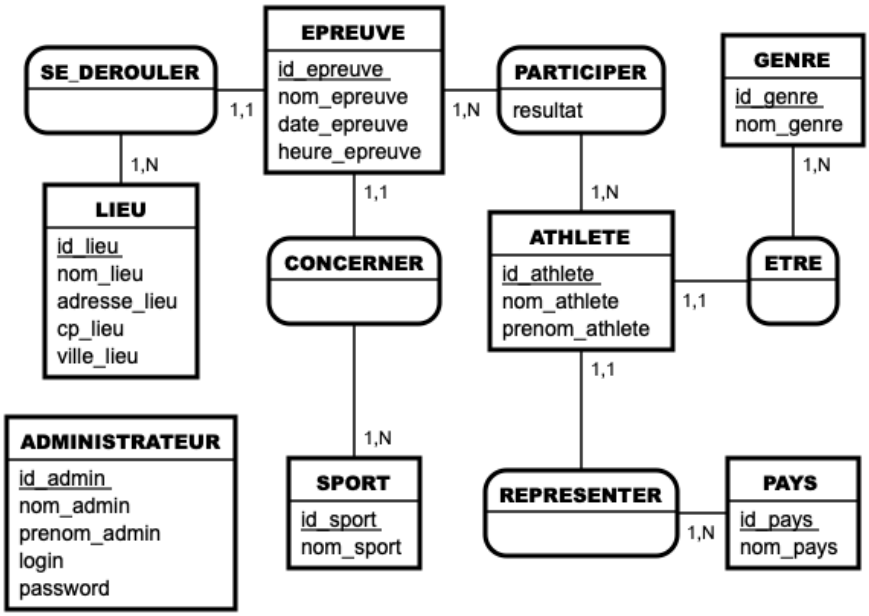


Figure 17 – Modèle Conceptuel de Données (MCD)

- Une **épreuve** se déroule dans un seul **lieu** et concerne un seul **sport** ; un lieu ou un sport peut accueillir plusieurs épreuves.
- Un **athlète** représente un seul **pays** et appartient à un seul **genre** ; un pays ou un genre regroupe plusieurs athlètes.

- Un **athlète** participe à une ou plusieurs **épreuves** et une épreuve réunit plusieurs athlètes : l'association PARTICIPER porte l'attribut « resultat ».
- Les **administrateurs** (ADMINISTRATEUR) accèdent à l'espace de gestion ; ils n'ont pas de lien avec les autres entités.

5.2.4. Modèle Logique de Données (MLD)

Le modèle logique est obtenu à partir du MCD : chaque entité devient une table, chaque association de cardinalité 1,1 donne une clé étrangère, et l'association PARTICIPER (1,N – 1,N) devient une table dont la clé primaire est composée des deux identifiants. Légende : les clés primaires sont soulignées, les clés étrangères sont précédées de #.

ATHLETE (id_athlete, nom_athlete, prenom_athlete, #id_pays, #id_genre)

EPREUVE (id_epreuve, nom_epreuve, date_epreuve, heure_epreuve, #id_lieu, #id_sport)

GENRE (id_genre, nom_genre)

LIEU (id_lieu, nom_lieu, adresse_lieu, cp_lieu, ville_lieu)

PARTICIPER (#id_athlete, #id_epreuve, resultat)

PAYS (id_pays, nom_pays)

SPORT (id_sport, nom_sport)

ADMINISTRATEUR (id_admin, nom_admin, prenom_admin, login, password)

Clés primaires et clés étrangères

Table	Clé primaire	Clés étrangères
ATHLETE	id_athlete	id_pays → PAYS ; id_genre → GENRE
EPREUVE	id_epreuve	id_lieu → LIEU ; id_sport → SPORT
PARTICIPER	id_athlete, id_epreuve	id_athlete → ATHLETE ; id_epreuve → EPREUVE
GENRE	id_genre	—
LIEU	id_lieu	—
PAYS	id_pays	—
SPORT	id_sport	—
ADMINISTRATEUR	id_admin	—

5.2.5. Modèle Physique de Données (MPD)

Le modèle physique correspond à la base de données MySQL « nom_prenom_jo2028 », créée à partir du script « db-jo.sql » et visualisée dans phpMyAdmin. Les identifiants sont des entiers auto-incrémentés, les clés étrangères garantissent l'intégrité des liens entre les tables et le mot de passe est stocké dans une colonne VARCHAR(255), suffisante pour un hachage Bcrypt.

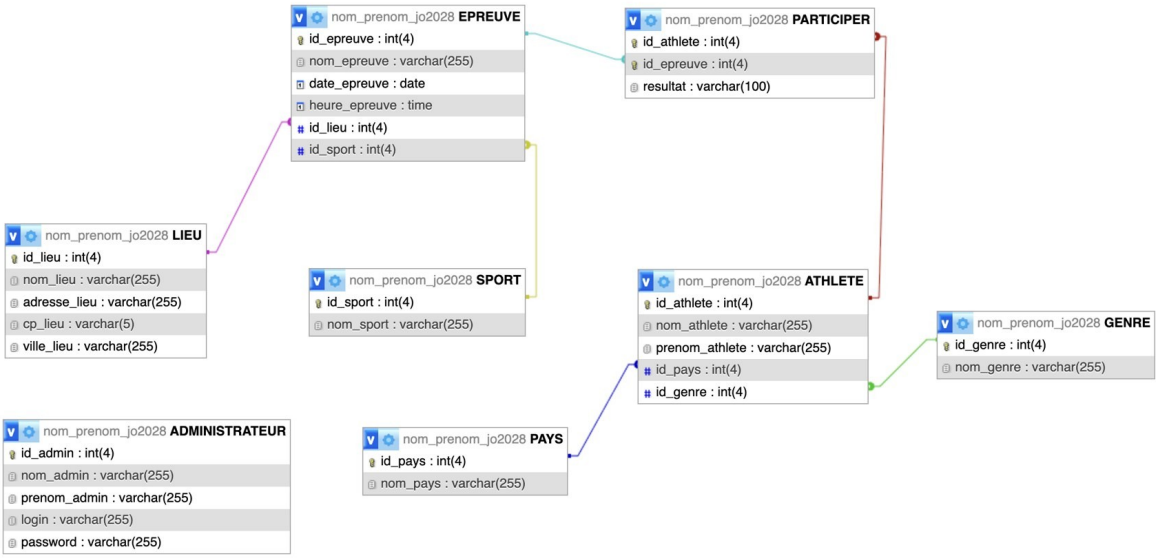


Figure 18 – Modèle Physique de Données (MPD)

6. Technologies utilisées

6.1. Langages de développement Web

- **Back-end** : PHP, avec l'extension PDO pour assurer une connexion sécurisée à la base de données.
- **Base de données** : SQL, pour la manipulation et l'interrogation de la base de données relationnelle MySQL.
- **Front-end** : HTML et CSS pour la structure et l'intégration de la charte graphique, complétés par JavaScript pour l'interactivité côté client uniquement.

6.2. Base de données

Les données sont stockées dans une **base de données relationnelle MySQL** de huit tables (ATHLETE, EPREUVE, GENRE, LIEU, PARTICIPER, PAYS, SPORT et ADMINISTRATEUR). La base est administrée avec phpMyAdmin en local, interrogée depuis PHP avec l'extension PDO, puis mise en ligne sur la plateforme gratuite InfinityFree avec le site.

7. Sécurité

7.1. Login et protection des pages administrateurs

L'administrateur saisit son login et son mot de passe dans un formulaire. Le script PHP recherche le login dans la table ADMINISTRATEUR avec une requête préparée, puis compare le mot de passe saisi au hachage enregistré avec password_verify(). Si les identifiants sont corrects, une **session PHP** est ouverte ; sinon un message d'erreur générique est affiché (sans préciser si c'est le login ou le mot de passe qui est faux), conformément au diagramme d'activités.

```php
<?php // login.php
session_start();
require 'connexion.php';   // connexion PDO ($pdo)

$stmt = $pdo->prepare(
    'SELECT id_admin, password FROM ADMINISTRATEUR WHERE login = :login'
);
$stmt->execute([':login' => $_POST['login']]);
$admin = $stmt->fetch(PDO::FETCH_ASSOC);

if ($admin && password_verify($_POST['password'], $admin['password'])) {
    session_regenerate_id(true);            // évite la fixation de session
    $_SESSION['id_admin'] = $admin['id_admin'];
    header('Location: admin/index.php');
    exit;
}
$erreur = 'Login et/ou mot de passe incorrect.';
```

Les pages réservées aux administrateurs incluent, **en première ligne**, un fichier de contrôle : si la session ne contient pas d'identifiant administrateur, l'utilisateur est redirigé vers la page de connexion. La déconnexion détruit la session (session_destroy()).

```
<?php // verif_session.php, à inclure en haut de chaque page admin
session_start();
if (!isset($_SESSION['id_admin'])) {
    header('Location: ../login.php');
    exit;
}
```

7.2. Hachage des mots de passe avec Bcrypt

Bcrypt est un algorithme de hachage conçu pour les mots de passe. Il est à **sens unique** (on ne peut pas retrouver le mot de passe à partir du hachage), il ajoute automatiquement un **sel aléatoire** (deux mots de passe identiques donnent deux hachages différents) et son **coût** de calcul est volontairement élevé, ce qui ralentit les attaques par force brute. Ainsi, même si la base de données est volée, les mots de passe ne sont pas lisibles.

En PHP, password_hash() avec PASSWORD_BCRYPT produit un hachage de 60 caractères commençant par « $2y$ » à stocker dans la colonne password (VARCHAR(255)). À la connexion, password_verify() compare le mot de passe saisi au hachage enregistré.

```
// Création d'un administrateur : hachage avant l'insertion en base
$hash = password_hash($_POST['password'], PASSWORD_BCRYPT);
$stmt = $pdo->prepare('INSERT INTO ADMINISTRATEUR
    (nom_admin, prenom_admin, login, password)
    VALUES (:nom, :prenom, :login, :mdp)');
$stmt->execute([':nom' => $nom, ':prenom' => $prenom,
                ':login' => $login, ':mdp' => $hash]);

// Vérification à la connexion : renvoie true ou false
password_verify($mdpSaisi, $hashEnBase);
```

7.3. Protection contre les attaques XSS (Cross-Site Scripting)

Définition : une attaque XSS consiste à injecter du code JavaScript dans une page (par exemple via un champ de formulaire), qui sera ensuite exécuté par le navigateur des autres visiteurs. L'attaquant peut ainsi voler une session, rediriger vers un site frauduleux ou modifier l'affichage de la page.

Solution en PHP : toute donnée issue d'un formulaire ou de la base de données est **échappée à l'affichage** avec htmlspecialchars(), qui transforme les caractères spéciaux (<, >, ", ') en entités HTML inoffensives. Les saisies sont également nettoyées côté serveur (trim()), et le cookie de session est protégé contre la lecture par JavaScript (option HttpOnly).

```
<!-- Affichage sécurisé d'une donnée -->
<td><?= htmlspecialchars($sport['nom_sport'], ENT_QUOTES, 'UTF-8') ?></td>

<?php // Cookie de session inaccessible depuis JavaScript
ini_set('session.cookie_httponly', 1);
```

7.4. Protection contre les injections SQL

Définition : une injection SQL consiste à saisir du code SQL dans un champ pour modifier la requête exécutée par le serveur. Avec une requête construite par concaténation, la saisie « ' OR '1'='1 » dans le champ login permettrait par exemple de contourner l'authentification, et d'autres saisies pourraient lire ou supprimer des données.

Solution en PHP : toutes les requêtes utilisent l'extension **PDO** avec des **requêtes préparées**. La requête et les données sont envoyées séparément au serveur MySQL, qui traite les valeurs comme de simples données et jamais comme du code. PDO est configuré pour lever des exceptions et utiliser de vraies requêtes préparées, et les identifiants reçus dans l'URL sont convertis en entiers.

```php
$dsn = 'mysql:host=localhost;dbname=nom_prenom_jo2028;charset=utf8mb4';
$pdo = new PDO($dsn, $user, $mdp, [
    PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_EMULATE_PREPARES => false,
]);

// Requête préparée avec paramètres nommés
$stmt = $pdo->prepare('UPDATE SPORT SET nom_sport = :nom WHERE id_sport = :id');
$stmt->execute([':nom' => $nom, ':id' => (int) $_GET['id']]);
```